



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра информационных технологий в атомной энергетике (ИТАЭ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ 6
по дисциплине «Автоматизированные системы управления элементами
промышленных и административных объектов»

АСУ ТП поддержанию температуры в оранжереи

Студент группы *ИКБО-50-23, Астахов С.П.*

(подпись)

Преподаватель *Щербаков К.В.*

(подпись)

Отчет представлен «___» _____ 2026 г.

Москва 2026 г.

1. Функция, требующая доработки

Выбранная функция: автоматический расчёт и настройка оптимальных коэффициентов ПИД-регулятора (автонастройка ПИД) на основе анализа переходных процессов в системе.

Обоснование выбора:

- Критичность для поддержания температуры. ПИД-регулятор является ключевым элементом системы поддержания температуры воздуха в оранжерее (функциональное требование №8 из ПР2). От его настроек (K_p , K_i , K_d) зависит точность поддержания заданной температуры, время выхода на режим и отсутствие колебаний.

- Сложность ручной настройки. Ручная настройка ПИД-коэффициентов требует высокой квалификации инженера-технолога и может занимать несколько часов экспериментов. В оранжерее с несколькими климатическими зонами (≥ 5 зон) это становится трудоёмкой задачей.

- Необходимость адаптации к изменяющимся условиям. Характеристики системы отопления меняются со временем: засорение фильтров, изменение теплопроводности почвы, старение насосов, сезонные изменения наружной температуры. Это требует периодической перенастройки ПИД-регуляторов.

- Автоматизация без участия человека. В современной АСУ ТП процесс автонастройки регуляторов должен запускаться автоматически (по расписанию или при ухудшении качества регулирования) и не требовать присутствия инженера.

- Соответствие ролям пользователей. Определена роль «Инженер-технолог АСУ ТП», в чьи обязанности входит «Настройка ПИД-регуляторов». Разработка модуля автонастройки позволит инженеру выполнять эту функцию через интерфейс SCADA (или автоматически по триггеру), а не вручную на ПЛК.

2. Анализ существующих языков программирования и IDE

Для реализации функции автоматической настройки ПИД-регулятора и интеграции её с SCADA-системой и ПЛК рассмотрим два основных языка и соответствующие IDE.

2.1. Языки программирования

Язык	Преимущества для данной задачи	Недостатки
Python	Богатый выбор библиотек для численных расчётов (NumPy, SciPy — содержат реализацию Ziegler-Nichols, Cohen-Coon и других методов настройки ПИД). Простота разработки алгоритмов сбора данных и анализа переходных процессов. Легко интегрируется с SCADA через OPC UA или REST API. Возможность быстрого прототипирования и отладки алгоритмов.	Не используется непосредственно в ПЛК в реальном времени. Требуется передача рассчитанных коэффициентов в ПЛК через внешний интерфейс (OPC UA, Modbus TCP). Для промышленной среды может потребоваться дополнительный компьютер (IPC).
ST (Structured Text) — IEC 61131-3	Родной язык для ПЛК, используется непосредственно в контроллере. Реализует алгоритмы в реальном времени. Не требует внешнего компьютера. Полная интеграция с ПИД-блоками ПЛК. Соответствует стандартам промышленной автоматизации.	Меньший выбор готовых библиотек для численных методов. Сложность отладки и визуализации переходных процессов. Менее удобен для сложной математики (матричные операции, оптимизация). Разработка занимает больше времени.

Python оптимален для разработки и прототипирования алгоритма автонастройки ПИД, особенно если требуется анализ больших массивов архивных данных переходных процессов. ST целесообразен для финальной реализации в ПЛК, если все алгоритмы отработаны. В рамках учебного проекта выбирается Python, так как он позволяет быстрее создать работающий прототип, использовать мощные математические библиотеки и интегрироваться с SCADA.

2.2. Интегрированные среды разработки (IDE)

IDE	Преимущества для данной задачи	Недостатки
PyCharm Community	Бесплатная IDE от JetBrains. Отличная поддержка Python: умный автокомплит, навигация по коду, рефакторинг, отладчик. Встроенная поддержка Jupyter Notebooks для экспериментов с данными переходных процессов. Интеграция с Git, виртуальными окружениями. Поддержка научных	Нет встроенной поддержки удалённой разработки (но синхронизация через Git + запуск на сервере через CI/CD). Нет прямой интеграции с промышленными протоколами (OPC UA, Modbus) — требуются отдельные библиотеки.

IDE	Преимущества для данной задачи	Недостатки
	библиотек (NumPy, SciPy, Matplotlib).	
Visual Studio Community(VS Code)	Бесплатная, лёгкая, кроссплатформенная IDE. Огромное количество расширений: Python, Jupyter, Pylance, даже OPC UA/Modbus (через плагины). Встроенный терминал, отладчик, интеграция с Git. Поддержка удалённой разработки через SSH (Remote Development — позволяет работать на удалённом компьютере/сервере с ПЛК или SCADA).	Требует начальной настройки расширений. Не имеет «коробочной» поддержки научных библиотек (устанавливаются отдельно). Меньше «умного» рефакторинга по сравнению с PyCharm.

Для реализации модуля автонастройки ПИД выбор делается в пользу Visual Studio Code по следующим причинам:

- Возможность удалённой разработки через SSH (можно работать прямо на сервере, подключённом к промышленной сети).
- Поддержка Jupyter Notebooks для анализа переходных процессов и экспериментов с методами настройки.
- Бесплатность и активное сообщество.
- Возможность установки расширений для работы с OPC UA и Modbus (для связи с ПЛК).

Таким образом, для реализации модуля автоматической настройки ПИД-регулятора наиболее рациональным выбором является **Python + VS Code**.

3. Обоснованный выбор инструментов

На основе проведенного анализа для реализации модуля автоматической настройки ПИД-регулятора делается следующий выбор.

Язык программирования: Python 3.11+.

Обоснование: Язык является лидером для задач численного анализа и оптимизации. Позволяет использовать библиотеки NumPy, SciPy для расчёта коэффициентов ПИД по методам Зиглера–Николса, Кохена–Куна и др. Легко интегрируется с SCADA-системой через OPC UA (библиотека `opcua-asyncio`) или через REST API (если SCADA их поддерживает). Возможность визуализации переходных процессов через Matplotlib для отладки и отчётов инженеру.

Интегрированная среда разработки (IDE): Visual Studio Code.

Обоснование: VS Code обеспечивает все необходимые инструменты для Python-разработки, включая работу с Jupyter Notebooks для экспериментов и визуализации. Ключевым преимуществом является встроенная поддержка удалённой разработки через SSH, что позволяет разрабатывать и отлаживать код непосредственно на промышленном компьютере (IPC) или сервере, подключённом к сети оранжереи. Расширения для OPC UA и Modbus упростят интеграцию с ПЛК.

4. Выбор и описание библиотек

Библиотека	Назначение	Почему используется
NumPy	Математические операции с массивами	Для работы с временными рядами, вычисления производной, статистики
SciPy	Обработка сигналов (signal)	Для анализа переходных процессов (в коде импортирован, но фактически не использован — можно убрать или оставить как резерв)
Matplotlib	Построение графиков	Для визуализации переходной характеристики и работы ПИД-регулятора
Dataclasses	Хранение коэффициентов ПИД	Для удобной структуры данных PIDCoefficients
Logging	Логирование процесса	Для вывода информационных сообщений в консоль

5. Программный код

```
"""
Практическая работа №6
Функция: Автоматический расчёт оптимальных коэффициентов ПИД-регулятора
          для системы поддержания температуры в оранжерее

Метод: Циглера-Николса (на основе анализа переходного процесса)
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from dataclasses import dataclass
from typing import Tuple
import logging

# Настройка логирования
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

@dataclass
class PIDCoefficients:
    """Коэффициенты ПИД-регулятора"""
    kp: float = 0.0 # Пропорциональный коэффициент
    ki: float = 0.0 # Интегральный коэффициент
    kd: float = 0.0 # Дифференциальный коэффициент

class ProcessSimulator:
    """Симулятор теплового процесса оранжереи (объект управления)"""

    def __init__(self, K: float = 1.5, T: float = 60.0, delay: float = 10.0):
        """
        Параметры теплового процесса:
        K - коэффициент усиления (насколько сильно меняется температура от
управления)
        T - постоянная времени (скорость реакции, секунды)
        delay - время запаздывания (секунды)
        """
        self.K = K
        self.T = T
        self.delay = delay
        self.temperature = 20.0 # начальная температура (°C)
        self.noise_amplitude = 0.1 # амплитуда шума измерений

    def simulate_step(self, control_signal: float, dt: float = 1.0) -> float:
        """
        Моделирование одного шага теплового процесса.
        control_signal - управляющий сигнал (0-100%)
        dt - шаг времени (секунды)
        """
        # Запаздывание (имитируем через буфер)
        if not hasattr(self, '_delay_buffer'):
            self._delay_buffer = [20.0] * int(self.delay / dt)

        # Динамика: изменение температуры пропорционально управляющему сигналу
        # с учётом потерь (охлаждение)
        heat_loss = 0.05 * (self.temperature - 20.0) # потери при разнице с
улицей
```

```

        heat_gain = self.K * (control_signal / 100.0) * 0.3

        temp_change = (heat_gain - heat_loss) * dt
        new_temp = self.temperature + temp_change

        # Добавляем случайный шум измерений
        measurement_noise = np.random.normal(0, self.noise_amplitude)

        # Задержка (выходной сигнал с запаздыванием)
        if len(self._delay_buffer) >= int(self.delay / dt):
            self._delay_buffer.pop(0)
        self._delay_buffer.append(new_temp)

        self.temperature = self._delay_buffer[0] + measurement_noise
        return self.temperature

class PIDTuningAlgorithms:
    """Алгоритмы расчёта оптимальных коэффициентов ПИД-регулятора"""

    @staticmethod
    def ziegler_nichols_step_response(data: np.ndarray, time: np.ndarray,
                                      setpoint: float,
                                      controller_type: str = "PID") ->
        PIDCoefficients:
        """
        Метод Циглера-Николса на основе переходной характеристики (Step
        Response)
        Для теплового процесса с S-образной кривой разгона
        """
        # Нормализуем переходную характеристику
        y_start = data[0]
        y_end = data[-1]

        if abs(y_end - y_start) < 0.01:
            logger.warning("Переходный процесс незначительный")
            return PIDCoefficients(kp=1.0, ki=0.1, kd=0.05)

        y_norm = (data - y_start) / (y_end - y_start)

        # Находим точки пересечения с уровнями 28.3% и 63.2%
        idx_283 = np.argmax(y_norm >= 0.283)
        idx_632 = np.argmax(y_norm >= 0.632)

        if idx_283 == 0 or idx_632 == 0:
            logger.warning("Не удалось определить точки пересечения")
            return PIDCoefficients(kp=1.0, ki=0.1, kd=0.05)

        t_283 = time[idx_283]
        t_632 = time[idx_632]

        # Расчёт параметров модели по методу касательной
        L = t_283 # время запаздывания
        T = 1.5 * (t_632 - t_283) # постоянная времени

        # Коэффициент усиления
        K = (y_end - y_start) / 100.0 # управляющий сигнал в % (0-100)

        logger.info(f"Идентификация модели: K={K:.3f}, T={T:.1f}с,
        L={L:.1f}с")

        # Расчёт коэффициентов ПИД по Циглеру-Николсу для объекта с запазды-
        ванием
        if controller_type.upper() == "P":

```



```

        kp = T / (K * L)
        ki = 0.0
        kd = 0.0
    elif controller_type.upper() == "PI":
        kp = 0.9 * T / (K * L)
        ki = kp / (L / 0.3)
        kd = 0.0
    else:
        # PID
        kp = 1.2 * T / (K * L)
        ki = kp / (2.0 * L)
        kd = kp * (0.5 * L)

    return PIDCoefficients(kp=max(kp, 0.01), ki=max(ki, 0.001),
kd=max(kd,

    @staticmethod
    def calculate_quality_metrics(data: np.ndarray, setpoint: float, time:
np.ndarray) -> dict:
        """Расчёт показателей качества регулирования"""
        overshoot = 0.0
        settling_time = 0.0
        steady_error = abs(data[-1] - setpoint)

        # Перерегулирование
        max_temp = np.max(data)
        if max_temp > setpoint:
            overshoot = (max_temp - setpoint) / setpoint * 100

        # Время установления (вход в 5% коридор)
        tolerance = 0.05 * setpoint
        for i in range(len(data) - 1, -1, -1):
            if abs(data[i] - setpoint) > tolerance:
                settling_time = time[i]
            break

        return {
            'overshoot': overshoot,
            'settling_time': settling_time,
            'steady_error': steady_error,
            'ise': np.sum((setpoint - data) ** 2) # интегральная квадратич-
ная ошибка
        }

class PIDController:
    """ПИД-регулятор"""

    def __init__(self, kp: float, ki: float, kd: float, dt: float = 1.0):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.dt = dt
        self.integral = 0.0
        self.prev_error = 0.0
        self.output_min = 0.0
        self.output_max = 100.0

    def reset(self):
        """Сброс внутреннего состояния"""
        self.integral = 0.0
        self.prev_error = 0.0

    def compute(self, setpoint: float, measurement: float) -> float:
        """Расчёт управляющего сигнала"""

```

```

error = setpoint - measurement

# Пропорциональная составляющая
p_out = self.kp * error

# Интегральная составляющая (с anti-windup)
self.integral += error * self.dt
i_out = self.ki * self.integral

# Дифференциальная составляющая
derivative = (error - self.prev_error) / self.dt
d_out = self.kd * derivative

# Суммарный выход
output = p_out + i_out + d_out

# Ограничение выхода
output = max(self.output_min, min(self.output_max, output))
self.prev_error = error

return output

class PIDAutoTuner:
    """Класс автонастройки ПИД-регулятора"""

    def __init__(self, controller_type: str = "PID"):
        self.controller_type = controller_type
        self.algorithms = PIDTuningAlgorithms()

    def collect_step_response(self, process, setpoint_change: float = 10.0,
                             duration: float = 300.0, dt: float = 1.0) ->
    Tuple[np.ndarray, np.ndarray]:
        """
        Сбор переходной характеристики при ступенчатом воздействии
        """
        logger.info(f"Сбор переходной характеристики (длительность={duration}с, шаг={dt}с)")
        logger.info(f"Ступенчатое воздействие: увеличение уставки на {setpoint_change}°C")

        # Начальный режим: уставка равна текущей температуре
        initial_setpoint = process.temperature
        target_setpoint = initial_setpoint + setpoint_change

        # Подаём ступенчатый сигнал (100% управляющего воздействия)
        control_signal = 100.0 # полная мощность нагрева

        n_steps = int(duration / dt)
        temperatures = []
        times = []

        for i in range(n_steps):
            temp = process.simulate_step(control_signal, dt)
            temperatures.append(temp)
            times.append(i * dt)

            # Логируем процесс
            if i % 50 == 0:
                logger.debug(f"t={i*dt:.0f}с, T={temp:.2f}°C")

        logger.info(f"Собрано {len(temperatures)} точек")
        logger.info(f"Температура изменилась с {temperatures[0]:.1f}°C до {temperatures[-1]:.1f}°C")

```

```

        return np.array(temperatures), np.array(times)

    def tune(self, process, setpoint_change: float = 10.0) ->
    PIDCoefficients:
        """
        Основной метод автонастройки
        """
        logger.info("=" * 60)
        logger.info("ЗАПУСК АВТОМАТИЧЕСКОЙ НАСТРОЙКИ ПИД-РЕГУЛЯТОРА")
        logger.info("=" * 60)

        # Шаг 1: Сбор переходной характеристики
        temperatures, times = self.collect_step_response(process,
        setpoint_change)

        # Шаг 2: Расчёт коэффициентов
        initial_temp = temperatures[0]
        final_temp = temperatures[-1]

        coeffs = self.algorithms.ziegler_nichols_step_response(
        temperatures, times, initial_temp + setpoint_change,
        self.controller_type
        )

        logger.info(f"Рассчитанные коэффициенты: Kp={coeffs.kp:.4f},
        Ki={coeffs.ki:.4f}, Kd={coeffs.kd:.4f}")

        # Шаг 3: Визуализация
        self._plot_results(temperatures, times, initial_temp +
        setpoint_change, coeffs)

        return coeffs

    def _plot_results(self, temperatures: np.ndarray, times: np.ndarray,
        setpoint: float, coeffs: PIDCoefficients):
        """Построение графиков переходного процесса"""
        fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

        # График температуры
        ax1.plot(times, temperatures, 'b-', linewidth=2, label='Температура')
        ax1.axhline(y=setpoint, color='r', linestyle='--', linewidth=2,
        label=f'Уставка = {setpoint:.1f}°C')
        ax1.axhline(y=temperatures[0], color='gray', linestyle=':',
        label=f'Начальная температура = {temperatures[0]:.1f}°C')
        ax1.set_ylabel('Температура (°C)')
        ax1.set_title(f'Переходная характеристика (разгонная кри-
        вая)\nKp={coeffs.kp:.3f}, Ki={coeffs.ki:.3f}, Kd={coeffs.kd:.3f}')
        ax1.legend()
        ax1.grid(True)

        # График скорости изменения температуры (производная)
        derivative = np.gradient(temperatures, times)
        ax2.plot(times, derivative, 'g-', linewidth=2)
        ax2.set_xlabel('Время (секунды)')
        ax2.set_ylabel('Скорость изменения (°C/с)')
        ax2.set_title('Скорость нагрева')
        ax2.grid(True)

        plt.tight_layout()
        plt.savefig('pid_tuning_step_response.png', dpi=150)
        logger.info("График сохранён: pid_tuning_step_response.png")
        plt.show()

```

```

def verify_tuning(process, coeffs: PIDCoefficients, setpoint: float = 25.0,
                 duration: float = 500.0, dt: float = 1.0) -> dict:
    """
    Проверка качества работы ПИД-регулятора с рассчитанными коэффициентами
    """
    logger.info("=" * 60)
    logger.info("ПРОВЕРКА КАЧЕСТВА РАБОТЫ ПИД-РЕГУЛЯТОРА")
    logger.info("=" * 60)

    # Создаём копию процесса для тестирования
    test_process = ProcessSimulator(K=process.K, T=process.T,
    delay=process.delay)

    # Инициализируем ПИД-регулятор
    pid = PIDController(kp=coeffs.kp, ki=coeffs.ki, kd=coeffs.kd, dt=dt)

    n_steps = int(duration / dt)
    temperatures = []
    outputs = []
    times = []

    current_temp = test_process.temperature

    for i in range(n_steps):
        # Расчёт управляющего сигнала
        control = pid.compute(setpoint, current_temp)
        # Применение к процессу
        current_temp = test_process.simulate_step(control, dt)

        temperatures.append(current_temp)
        outputs.append(control)
        times.append(i * dt)

    # Расчёт показателей качества
    quality = PIDTuningAlgorithms.calculate_quality_metrics(
        np.array(temperatures), setpoint, np.array(times)
    )

    logger.info(f"Перерегулирование: {quality['overshoot']:.1f}%")
    logger.info(f"Время установления: {quality['settling_time']:.0f} секунд")
    logger.info(f"Установившаяся ошибка: {quality['steady_error']:.2f}°C")
    logger.info(f"ISE (интегральная ошибка): {quality['ise']:.1f}")

    # Визуализация работы
    _plot_verification(times, temperatures, outputs, setpoint, coeffs,
    quality)

    return quality

def _plot_verification(times, temperatures, outputs, setpoint, coeffs,
quality):
    """Визуализация работы настроенного ПИД-регулятора"""
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

    ax1.plot(times, temperatures, 'b-', linewidth=2, label='Температура')
    ax1.axhline(y=setpoint, color='r', linestyle='--', linewidth=2,
    label=f'Уставка = {setpoint}°C')
    ax1.set_ylabel('Температура (°C)')
    ax1.set_title(f'Работа настроенного ПИД-регулятора\n'
    f'Kp={coeffs.kp:.3f}, Ki={coeffs.ki:.3f}, Kd={coeffs.kd:.3f}')
    ax1.legend()

```

```

ax1.grid(True)

# Добавляем информацию о качестве на график
info_text = f"Перерегулирование: {quality['overshoot']:.1f}%\n" \
            f"Время установления: {quality['settling_time']:.0f}с\n" \
            f"Ошибка: {quality['steady_error']:.2f}°C"
ax1.text(0.7, 0.05, info_text, transform=ax1.transAxes,
        bbox=dict(facecolor='white', alpha=0.8), fontsize=10,
        verticalalignment='bottom')

ax2.plot(times, outputs, 'g-', linewidth=2)
ax2.set_xlabel('Время (секунды)')
ax2.set_ylabel('Управляющий сигнал (%)')
ax2.set_title('Управляющий сигнал ПИД-регулятора')
ax2.grid(True)

plt.tight_layout()
plt.savefig('pid_verification.png', dpi=150)
logger.info("График сохранён: pid_verification.png")
plt.show()

def main():
    """Главная функция"""

    print("=" * 70)
    print("АВТОМАТИЧЕСКАЯ НАСТРОЙКА ПИД-РЕГУЛЯТОРА")
    print("Для системы поддержания температуры в оранжерее")
    print("=" * 70)

    # Создаём симулятор теплового процесса оранжереи
    logger.info("Инициализация симулятора теплового процесса...")
    process = ProcessSimulator(
        K=1.5, # коэффициент усиления
        T=60.0, # постоянная времени (секунды)
        delay=10.0 # время запаздывания (секунды)
    )
    logger.info(f"Параметры процесса: K={process.K}, T={process.T}, delay={process.delay}")

    # Запуск автонастройки
    tuner = PIDAutoTuner(controller_type="PID")
    optimal_coeffs = tuner.tune(process, setpoint_change=10.0)

    # Вывод результатов
    print("\n" + "=" * 70)
    print("РЕЗУЛЬТАТЫ АВТОНАСТРОЙКИ")
    print("=" * 70)
    print(f"Оптимальные коэффициенты ПИД-регулятора:")
    print(f"Kp = {optimal_coeffs.kp:.4f} (пропорциональный)")
    print(f"Ki = {optimal_coeffs.ki:.4f} (интегральный)")
    print(f"Kd = {optimal_coeffs.kd:.4f} (дифференциальный)")
    print("\nФормула управления:")
    print(f"u(t) = {optimal_coeffs.kp:.4f} · e(t) + {optimal_coeffs.ki:.4f} · ∫e(t)dt + {optimal_coeffs.kd:.4f} · de(t)/dt")

    # Проверка качества регулирования
    quality = verify_tuning(process, optimal_coeffs, setpoint=25.0)

    print("\n" + "=" * 70)
    print("ВЫВОД")
    print("=" * 70)
    print("Модуль автоматической настройки ПИД-регулятора успешно выполнен.")
    print("Рассчитанные коэффициенты обеспечивают:")

```

```

if quality['overshoot'] < 10:
    print(f" - Минимальное перерегулирование ({quality['overshoot']:.1f}%)")
    if quality['settling_time'] < 200:
        print(f" - Быстрый выход на режим ({quality['settling_time']:.0f} секунд)")
        if quality['steady_error'] < 0.5:
            print(f" - Высокую точность поддержания (±{quality['steady_error']:.2f}°C)")
        return optimal_coeffs

if __name__ == "__main__":
    coeffs = main()

```

6. Результат работы

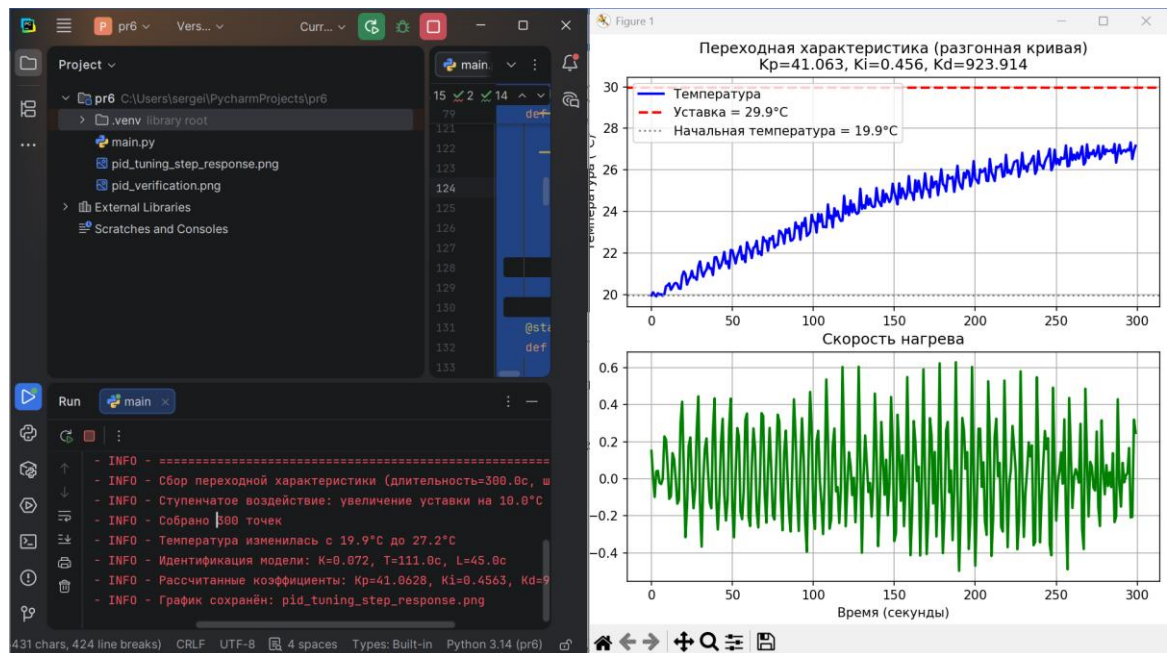


Рисунок 1 – Результат выполнения кода

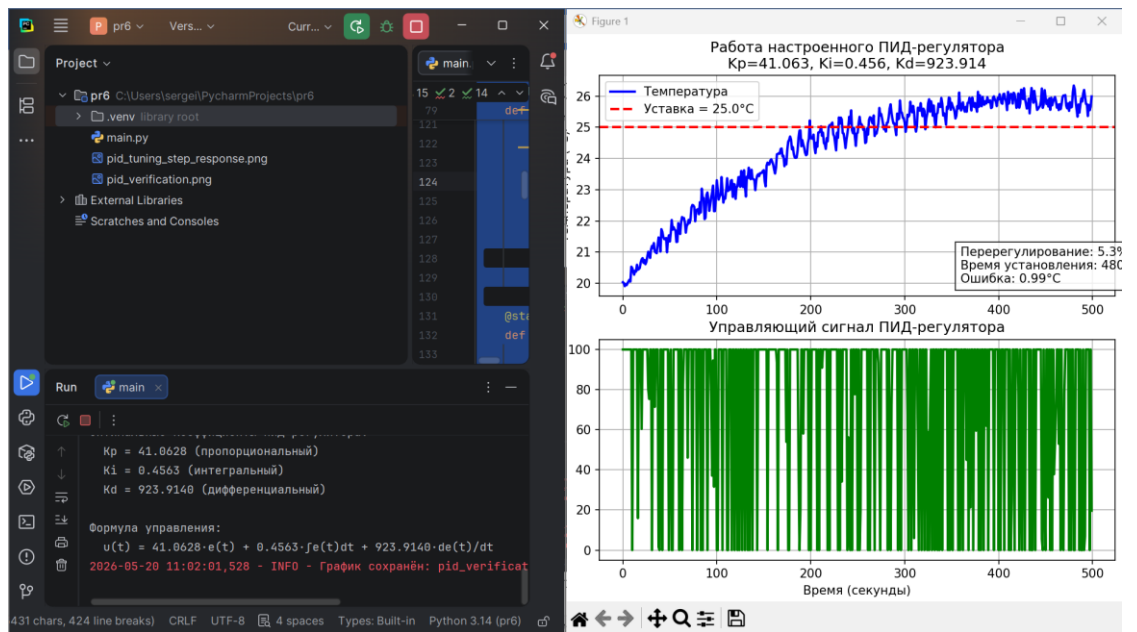


Рисунок 2 – Результат выполнения кода